

Topics: will going to discuss here

login validation, What is a Session? Use of Sessions, Types of Sessions, Session vs. Cookies, Session Configuration in Flask, Creating a Session, Session Validation (User Login Check) Session Destruction (Logout), Session Expiry Handling dashboard jinja Inheritance

Consider we are creating a small **Login** → **Dashboard** → **Logout** mini app.

✓ What is Login Validation?

Login validation means checking whether the **username and password entered by the user are correct**.

For example:

- User enters username = "admin" and password = "1234"
 - We compare with stored data (in database or in code)
 - If matched → allow user to login
 - If not matched → show error message: *"Invalid credentials"*
-

✓ What is a Session?

A **Session** is a **temporary storage** used to **remember the user** while they browse your website.

Think like:

"Who is currently logged in?"

When a user logs in, we store their identity (like username or user_id) in a session. So on every page we know **this user is logged in**.

✓ Use of Sessions

- To **keep user logged in**
- To display **user-specific dashboard**
- To **restrict pages** (only logged users can see dashboard)

- Commonly used in **Login systems**
-

✓ Session vs Cookies

Feature	Session	Cookies
Stored in	Server side	Browser (Client)
Security	More secure	Less secure (can be modified)
Expiry	Automatically ends	Can be long-lasting
Speed	Slightly slower	Faster

✓ Types of Sessions

In Flask we mainly use:

1. **Server-side Session** (default using secret key)
2. **Client-side Session** (Flask stores session in encrypted cookies)

For beginners, just understand:

Flask Session is stored securely and managed by Flask using **secret_key**.

✓ Session Configuration in Flask

We must set a **secret key** to use sessions:

```
from flask import Flask, session
```

```
app = Flask(__name__)
```

```
app.secret_key = "mysecret123" # Used to encrypt session data
```

✓ Creating a Session (After Login)

When login is successful, we store username in session:

```
session['username'] = username
```

✓ Session Validation (Check user logged-in or not)

Before accessing dashboard:

if 'username' in session:

 # user is logged in

else:

 redirect to login page

✓ Session Destruction (Logout)

session.pop('username', None)

✓ Session Expiry Handling

We can set session expiry time using:

from datetime import timedelta

app.permanent_session_lifetime = timedelta(minutes=5)

session.permanent = True

It means session expires after **5 minutes** of inactivity.

★ Full Working Example with Templates (Easy)

Project Folder Structure

project/

├── app.py

└── templates/

 ├── base.html

 ├── login.html

 └── dashboard.html

✓ app.py

from flask import Flask, render_template, request, redirect, session

```
from datetime import timedelta
```

```
app = Flask(__name__)
```

```
app.secret_key = "mysecret123"
```

```
app.permanent_session_lifetime = timedelta(seconds=30)
```

```
# Hardcoded username & password
```

```
USER = {"username": "admin", "password": "1234"}
```

```
@app.route("/", methods=["GET", "POST"])
```

```
def login():
```

```
    if request.method == "POST":
```

```
        username = request.form['username']
```

```
        password = request.form['password']
```

```
        if username == USER["username"] and password == USER["password"]:
```

```
            session.permanent = True
```

```
            session['username'] = username
```

```
            return redirect("/dashboard")
```

```
        else:
```

```
            return "Invalid Credentials. Try Again!"
```

```
    return render_template("login.html")
```

```
@app.route("/dashboard")
```

```
def dashboard():
```

```
    if 'username' in session:
```

```

        return render_template("dashboard.html", user=session['username'])
    else:
        return redirect("/")

@app.route("/logout")
def logout():
    session.pop('username', None)
    return redirect("/")

if __name__ == "__main__":
    app.run(debug=True)

```

✓ Jinja Template Inheritance (Dashboard UI)

base.html

```

<!DOCTYPE html>

<html>

<head>

    <title>{% block title %}Flask Site{% endblock %}</title>

</head>

<body>

    <h1>My Flask Website</h1>

    <hr>

    {% block content %}{% endblock %}

</body>

</html>

```

login.html

```
{% extends 'base.html' %}

{% block title %}Login Page{% endblock %}


{% block content %}

<h2>Login</h2>

<form action="/" method="POST">

    Username: <input type="text" name="username"><br><br>

    Password: <input type="password" name="password"><br><br>

    <button type="submit">Login</button>

</form>

{% endblock %}
```

dashboard.html

```
{% extends 'base.html' %}

{% block title %}Dashboard{% endblock %}


{% block content %}

<h2>Welcome {{ user }}</h2>

<a href="/logout">Logout</a>

{% endblock %}
```

Output Flow

1. Open browser → login page
 2. Enter admin / 1234
 3. Redirected to Dashboard → shows username
 4. Click Logout → returns to login page
 5. Session expires automatically after 2 minutes of inactivity
-

✓ Summary (In Simple Words)

Feature	Meaning
Session	Remembers the logged-in user
Login Validation	Checking correct username & password
Session Validation	Checking whether user is logged in before showing dashboard
Logout	Removing user session
Jinja Inheritance	Avoid code repetition using base.html

Explanation Of Code:

`from flask import Flask, render_template, request, redirect, session`

`from datetime import timedelta`

We are importing the required modules:

Flask → Main framework

`render_template` → To load HTML pages

`request` → To get form data (username/password)

`redirect` → To move to another page

`session` → To store user login information

`timedelta` → To set session expiry time

`app = Flask(__name__)`

Creating Flask application object.

```
app.secret_key = "mysecret123"
```

Secret key is needed to use session securely.

It encrypts session data so others cannot read or modify it.

```
app.permanent_session_lifetime = timedelta(minutes=2)
```

This means session will expire after 2 minutes of inactivity.

```
# Hardcoded username & password
```

```
USER = {"username": "admin", "password": "1234"}
```

This is dummy login data.

In real projects, we get this from a database.

```
#Login Page Route
```

```
@app.route("/", methods=["GET", "POST"])
```

```
def login():
```

Route / is the first page (login page).

It accepts both GET (show page) and POST (form submit).

```
if request.method == "POST":
```

If the user clicks Login button, method will be POST.


```
username = request.form['username']
```

```
password = request.form['password']
```

We take the entered username and password from the form.

```
if username == USER["username"] and password == USER["password"]:
```

Checking if entered credentials match the stored values.

```
session.permanent = True
```

This tells Flask to apply expiry time to the session.

```
session['username'] = username
```

Storing username in session → Means user is now logged in.

```
return redirect("/dashboard")
```

Redirect user to Dashboard page after successful login.

```
else:
```

```
return "Invalid Credentials. Try Again!"
```

If username/password are wrong, show error.

```
return render_template("login.html")
```

If GET request → Just display the login page.

Dashboard Route

```
@app.route("/dashboard")
```

```
def dashboard():
```

This is the dashboard page, visible only after login.

```
if 'username' in session:
```

Checking if user is logged in.

If session contains username, user is allowed.

```
    return render_template("dashboard.html", user=session['username'])
```

Show dashboard page and pass username to HTML.

```
else:
```

```
    return redirect("/")
```

If session is empty → User is not logged in → Send to login page.

#Logout Route

```
@app.route("/logout")
```

```
def logout():
```

This route logs the user out.

```
    session.pop('username', None)
```

Remove username from session.

User is now logged out.

```
    return redirect("/")
```

Send user back to login page.

Run Application

```
if __name__ == "__main__":
```

```
    app.run(debug=True)
```

Run Flask app in debug mode, so errors will be shown properly.

Summary in Simple Words

Step	Meaning
------	---------

Login Page	User enters username & password
------------	---------------------------------

Validate	Check if correct
----------	------------------

Create SessionStore	username in session when login successful
---------------------	---

Dashboard	Check session before showing dashboard
-----------	--

Logout	Remove session → user logged out
--------	----------------------------------

Session Expiry	Auto logout after 2 minutes
----------------	-----------------------------

Code Examplation in simple way:

Importing required modules from Flask

```
from flask import Flask, render_template, request, redirect, session
```

```
from datetime import timedelta # Used to set session expiry time
```

Creating Flask application

```
app = Flask(__name__)
```

Secret key is required to use sessions in Flask

```
# It secures session data (encrypts it)
```

```
app.secret_key = "mysecret123"
```

This sets how long the session should stay active (2 minutes here)

app.permanent_session_lifetime = timedelta(minutes=2)

Hardcoded username and password (instead of using a database)

USER = {"username": "admin", "password": "1234"}

-----

LOGIN PAGE

-----

@app.route("/", methods=["GET", "POST"])

def login():

If form is submitted (POST request)

if request.method == "POST":

**# Fetching username and password entered by the user from
the form**

username = request.form['username']

password = request.form['password']

**# Checking if entered credentials match the stored/login
credentials**

**if username == USER["username"] and password ==
USER["password"]:**

session.permanent = True means session will use the expiry time defined above

session.permanent = True

Storing username in session (This means user is now logged in)

session['username'] = username

Redirect user to the dashboard page after successful login

return redirect("/dashboard")

else:

If credentials do not match, show error message

return "Invalid Credentials. Try Again!"

If GET request, show login page

return render_template("login.html")

-----

DASHBOARD PAGE (Protected Route)

-----

@app.route("/dashboard")

def dashboard():

```
# Checking if user is logged in by seeing if username exists in
session

if 'username' in session:

    # Pass username to dashboard.html for display

    return render_template("dashboard.html",
user=session['username'])

else:

    # If session is empty → user not logged in → send back to login
page

    return redirect("/")
```

```
# -----
```

```
# LOGOUT PAGE
```

```
# -----
```

```
@app.route("/logout")
```

```
def logout():
```

```
    # Removing username from session → logging out user
```

```
    session.pop('username', None)
```

```
    # Redirect user back to login page
```

```
    return redirect("/")
```

Running the application

```
if __name__ == "__main__":
```

```
    app.run(debug=True) # debug=True helps in showing errors  
clearly during development
```

😊 Simple Explanation

Part	Meaning
Login	User enters username & password
Session Created	If login is correct, username is stored in session
Dashboard Access	Only allowed if session contains username
Logout	Removes session and user is logged out
Session Expiry	Automatically logs out user after 2 minutes

N

We will use MySQL and mysql.connector library.

✅ Step 1: Install MySQL Connector

Open terminal / cmd:

```
pip install mysql-connector-python
```

✅ Step 2: Create Database & Table in MySQL

Open MySQL Workbench / CMD and run:

```
CREATE DATABASE userdb;
```

```
USE userdb;
```

```
CREATE TABLE users(  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    username VARCHAR(50),  
    password VARCHAR(50)  
);
```

```
-- Insert sample user
```

```
INSERT INTO users(username, password) VALUES ("admin",  
"1234");
```

✅ Step 3: Folder Structure

project/

├── app.py

└── templates/

├── login.html

└── dashboard.html

✅ Step 4: Updated app.py With Database Login + Session

```
from flask import Flask, render_template, request, redirect, session
```

```
import mysql.connector
```

```
from datetime import timedelta
```

```
app = Flask(__name__)
```

```
app.secret_key = "mysecret123"
```

```
app.permanent_session_lifetime = timedelta(minutes=2)
```

```
# Function to connect to MySQL database
```

```
def get_db_connection():
```

```
    return mysql.connector.connect(
```

```
        host="localhost",
```

```
        user="root",    # Change if your MySQL username is different
```

```
        password="",    # Enter your MySQL password
```

```
        database="userdb"
```

```
    )
```

```
# -----
```

```
# LOGIN PAGE
```

```
# -----
```

```
@app.route("/", methods=["GET", "POST"])
```

```
def login():
```

```
if request.method == "POST":  
    username = request.form['username']  
    password = request.form['password']  
  
    # Connecting to database  
    conn = get_db_connection()  
    cursor = conn.cursor()  
  
    # Query to check if username & password exists in database  
    cursor.execute("SELECT * FROM users WHERE username=%s  
AND password=%s", (username, password))  
    user = cursor.fetchone()  
  
    # Closing database connection  
    cursor.close()  
    conn.close()  
  
    if user: # If user record found  
        session.permanent = True  
        session['username'] = username  
        return redirect("/dashboard")  
    else:  
        return "Invalid Username or Password. Try Again!"
```

```
return render_template("login.html")
```

```
# -----
```

```
# DASHBOARD PAGE (Protected Route)
```

```
# -----
```

```
@app.route("/dashboard")
```

```
def dashboard():
```

```
    if 'username' in session:
```

```
        return render_template("dashboard.html",  
user=session['username'])
```

```
    else:
```

```
        return redirect("/")
```

```
# -----
```

```
# LOGOUT PAGE
```

```
# -----
```

```
@app.route("/logout")
```

```
def logout():
```

```
    session.pop('username', None)
```

```
    return redirect("/")
```

```
if __name__ == "__main__":  
    app.run(debug=True)
```

✓ Step 5: login.html

```
<!DOCTYPE html>  
  
<html>  
  
<head>  
    <title>Login</title>  
</head>  
  
<body>  
  
<h2>Login Page</h2>  
  
<form method="POST">  
  
    Username: <input type="text" name="username"  
required><br><br>  
  
    Password: <input type="password" name="password"  
required><br><br>  
  
    <button type="submit">Login</button>  
  
</form>  
  
</body>  
  
</html>
```

✓ Step 6: dashboard.html

```
<!DOCTYPE html>

<html>

<head>

  <title>Dashboard</title>

</head>

<body>

<h2>Welcome {{ user }}</h2>

<a href="/logout">Logout</a>

</body>

</html>
```

🎯 What Changed?

Before	Now
Username & password were fixed in code	Stored in MySQL Database
Login check was manual	Login check uses SQL query
No scalability	Can handle many users

★ Next Step (Optional but Recommended)

We can hash passwords instead of storing plain password (using `werkzeug.security`).

If you'd like, I will teach:

- `generate_password_hash()`
- `check_password_hash()`
- Secure Login System

Just tell me:

Do you want secure hashed password version? (Yes / No) 😊